Méthodes numériques

Projet I

Circuit électrique

Licence L3 Physique
CY Cergy Paris Université
semy.el-karra@etu.cyu.fr

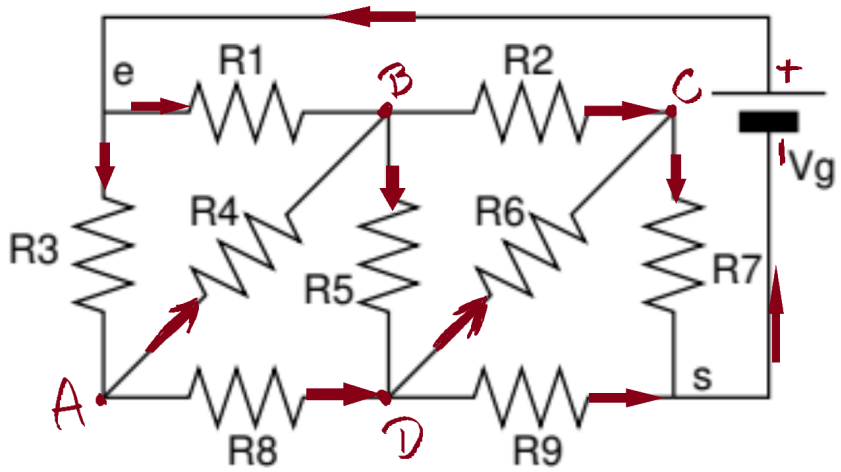
11 septembre 2025



Résumé

Dans ce rapport nous allons aborder la résolution d'un systèmes d'équations linaires appliqué à un circuit électrique alimenté par un générateur idéale de tensionet 9 résisteurs.Nous allons donc résoudre une équation matricielle Ax = b et la résoudre numériquement grace au procéder d'élimination de Gauss. Nous allons donc expliciter et détailler dans ce rapport les différentes méthodes à appliquer afin de trouver le courant fournit par le générateur et la résistances équivalente entre l'entrée et la sortie. Les calculs ont été effectué sur python et vérifiés par `numpy.linalg.solve`.

I Problématique



À partir de ce schéma du circuit muni de 9 résistances $R_1, \ldots, R_9$ et une source idéale V_g), on souhaite déterminer les courants $I_1, \ldots, I_9$ en utilisant :

$$Ax = b, \qquad x = (I_1, \ldots, I_9)^\top.$$

Les objectifs du rapport sont :

- Etablir les équations Kirchhoff et vérifier leur **indépendance**
- Mettre sous la formes matricielle $Ax = b$(Remarque nous aurions pu travailler avec la matrice $[A|b]$)
- Facultatif mais on peut vérifier la condition d'inversibilité (dét non nul | rang =max(n,m) de la matrice A avec `numpy.linalg.solve` permet de savoir à l'avance si notre système est bien indépendant ;
- Utiliser et expliquerl'algorithme de Gauss avec pivot partiel et la rétro-substitution ;
- Comparer la solution obtenue avec l'algorithme avec la solution fournie par `numpy.linalg.solve` ;
- Calculer le coût opératoire.

II Données de la problématique

Les résistances et la source (valeurs fournies) :

$$(R_1, \ldots, R_9) = (2.5,\ 10.0,\ 5.1,\ 8.2,\ 7.9,\ 3.6,\ 3.3,\ 3.0,\ 7.1)\ \text{k}\Omega,$$

on travaillera en Ω :

$$R_k \leftarrow 1000 \cdot R_k \quad (\Omega).$$

La source : $V_g = 80\,\text{V}$.

III Etablir les lois de Kirchhoff :

On applique les lois de Kirchhoff pour obtenir un système linéaire Ax = b Les équations de nœuds etles équations de mailles doivent être indépendantes afin de garantir que A est bien inversible

et ne pas se retrouver avec un pivot null. Nous avons 9 inconnues ils nous faut donc au moins 9 équations.

Donc nous partirons sur les équations suivantes :

1. **Mailles (4 équations)** :Les 4 mailles sont indépendantes couvrant le circuit.
2. **Nœuds (4 équations)** : Nous utiliserons 4 noeuds sur 5 car la 5e est tout simplement une combinaison linaire des autres.
3. **Équation générateur des tensions** : Nous permts de compléter nos 9 équations indépendantes.

IV Sytème linaire

En notant les courants d'orientation choisie $I_1, \ldots, I_9$, sur le shéma nous pouvons déduire le système suivant :

Loi des Mailles :

$$\begin{aligned}
\text{Maille 1 :} \quad & R_1 I_1 - R_3 I_3 + R_5 I_5 - R_8 I_8 = 0, \\
\text{Maille 2 :} \quad & R_2 I_2 - R_5 I_5 + R_7 I_7 - R_9 I_9 = 0, \\
\text{Maille 3 :} \quad & R_2 I_2 + R_4 I_4 - R_6 I_6 - R_8 I_8 = 0, \\
\text{Maille 4 :} \quad & R_1 I_1 + R_2 I_2 + R_7 I_7 = V_g. \\
\text{Maille 5 :} \quad & R_6 I_6 + R_7 I_7 - R_9 I_9 = 0
\end{aligned}$$

Loi des Nœuds :

$$\begin{aligned}
\text{Nœud A :} \quad & I_3 - I_4 - I_8 = 0, \\
\text{Nœud B :} \quad & I_1 - I_2 + I_4 - I_5 = 0, \\
\text{Nœud C :} \quad & I_2 + I_6 - I_7 = 0, \\
\text{Nœud D :} \quad & I_5 - I_6 + I_8 - I_9 = 0.
\end{aligned}$$

V Forme matricielle $Ax = b$

On pose $x = (I_1, \ldots, I_9)^\top$. La matrice A s'écrit de cette manière, nous pouvons faire le produit de matrice et retomber sur nos équations au dessus :

$$A_{\text{sym}} = \begin{pmatrix}
R_1 & 0 & -R_3 & 0 & R_5 & 0 & 0 & -R_8 & 0 \\
0 & R_2 & 0 & 0 & -R_5 & 0 & R_7 & 0 & -R_9 \\
0 & R_2 & 0 & R_4 & 0 & -R_6 & 0 & -R_8 & 0 \\
R_1 & R_2 & 0 & 0 & 0 & 0 & R_7 & 0 & 0 \\
0 & 0 & 1 & -1 & 0 & 0 & 0 & -1 & 0 \\
1 & -1 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & -1 & 0 & 1 & -1 \\
0 & 0 & 0 & 0 & 0 & R6 & R7 & 0 & -R9
\end{pmatrix}, \qquad b = \begin{pmatrix}
0 \\ 0 \\ 0 \\ V_g \\ 0 \\ 0 \\ 0 \\ 0 \\ 0
\end{pmatrix}.$$

En valeur numérique (valeurs en Ω) avec $R_1 = 2500$, $R_2 = 10000$, $R_3 = 5100$, $R_4 = 8200$, $R_5 = 7900$, $R_6 = 3600$, $R_7 = 3300$, $R_8 = 3000$, $R_9 = 7100$

Dans notre système, la matrice A peut contenir des équations issues de lois physiques différentes (par exemple des lois des nœuds exprimées en ampères et des lois des mailles exprimées en volts). Chaque ligne reste homogène en unités, car ses coefficients sont choisis pour que le produit Ax ait la même dimension physique que le second membre b de l'équation correspondante. La méthode de résolution (élimination de Gauss et rétro-substitution) manipule ces coefficients de manière purement algébrique ; elle ne modifie pas les unités, mais conserve la cohérence dimensionnelle.

$$A = \begin{pmatrix} 2500 & 0 & -5100 & 0 & 7900 & 0 & 0 & -3000 & 0 \\ 0 & 10000 & 0 & 0 & -7900 & 0 & 3300 & 0 & -7100 \\ 0 & 10000 & 0 & 8200 & 0 & -3600 & 0 & -3000 & 0 \\ 2500 & 10000 & 0 & 0 & 0 & 0 & 3300 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & 0 & -1 & 0 \\ 1 & -1 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & -1 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 & 0 & 3600 & 3300 & 0 & -7100 \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 80 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

VI Vérification de la matrice

Par précaution, nous allons vérifier si la martice obtenue A est bien inversible. Celà nous permettra d'isoler les potentiels problèmes.

- le **déterminant** : si $\det(A) = 0$ non inversible ;
- le **rang** : il doit être égal à (n=9) pour l'unicité de la solution ;
- le **conditionnement** :

  Les calculs effectués avec le module NumPy donnent :

  $$\det(A) \approx 1.035 \times 10^{20}, \qquad \text{rang}(A) = 9$$

  La matrice est donc **inversible**.

  Cependant le nombre de conditionnement de la matrice A est donné par :

  $$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}} = 5.9224 \times 10^4$$

  où :
  - $\sigma_{\max} = 3.416 \times 10^5$ est la plus grande valeur singulière
  - $\sigma_{\min} = 5.360$ est la plus petite valeur singulière

  Ce conditionnement élevé :

  $$\kappa(A) \approx 59\,224$$

  La matrice obtenue est **mal conditionnée**, ce qui explique les difficultés numériques que j'ai rencontrées lors de la résolution du système.Donc les résultats obtenus sont potentiellement inexacts. Après plusieurs jours et de nombreuses tentatives, je n'ai pas réussi à avoir un système qui satisfait le conditionnement. Cependant j'ai choisi de poursuivre avec mes propres équations, sans recourir aux données de mes camarades, afin de conserver authenticité de mon travail mais si il semble un échec.

  A savoir que les résultat obtenues avec les algorithmes sont exactement les même que ceux de la fonction numpy. Donc le problème ne vient certainement pas du programme en lui même.

VII Algorithmes utilisés

On applique :

1. **Algorithme 1.3 (Élimination de Gauss avec pivot partiel)** : pour trianguliariser la matrice en évitant les pivots faibles.
2. **Algorithme 1.2 (Rétro-substitution)** : pour résoudre le système triangulaire obtenu.

Remarque : la normalisation des coefficients n'est pas impérative pour la résolution du système. Elle peut faciliter l'interprétation.

Justification

Le pivot partiel (choix du pivot de plus grande valeur absolue dans la colonne $a[k][k]$) est impératif garantir la stabilité numérique et éviter des divisions par des nombres très petits.

Remarque : En soit on peut normaliser chaque ligne pour réduire la taille ou la grandeur des coefficients mais on va pas s'embêter avec.

•Résolution par élimination de Gauss avec la méthode du pivot partiel, on obtient donc $Tx = b'$

$T \in \mathcal{T}_n$ où $\mathcal{T}_n$ désigne l'ensemble des matrices triangulaires supérieures de taille $n \times n$

•Normalisation(facultatif) mais permets que les plus grands coifficients soient d'ordres 1.

•Rétro-substitution Mise à jour du vecteur b. À chaque échange de lignes dans la matrice A, il faut également échanger les composantes correspondantes du vecteur b. Lors de l'élimination, pour une ligne $i > k$, on calcule le coefficient :

$$h = \frac{A_{ik}}{A_{kk}}$$

puis on met à jour la composante b_i selon :

$$b_i \leftarrow b_i - h \cdot b_k$$

2 ème méthode : travailler avec la matrice$[A \mid b]$, et appliquer les mêmes opérations de lignes à tout le bloc. Cela garantit la cohérence des mises à jour entre A et b.

Pivot de Gauss

L'algorithme va suivre ces étapes suivantes:

1. Recherche du pivot maximal dans la k ème colonne.
2. Échange de lignes afin de placer le pivot choisit sur la diagonale.
3. Élimination des coefficients situés en dessous du pivot.
4. Répétitio pour chaque colonne de la matrice.

Exemple d'élimination de Gauss sur une matrice 4×4

Système initial

$$\begin{pmatrix} 2 & 1 & -1 & 3 \\ 4 & 4 & 1 & 2 \\ 6 & 7 & 2 & -1 \\ 2 & 5 & 3 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 1 \\ 4 \\ 3 \\ 2 \end{pmatrix}$$

Étape 1 : Recherche du pivot dans la colonne 1

Pivot maximal: $|a_{31}| = 6$ (ligne 3)

$$\begin{pmatrix} 2 & 1 & -1 & 3 \\ 4 & 4 & 1 & 2 \\ \mathbf{6} & 7 & 2 & -1 \\ 2 & 5 & 3 & 4 \end{pmatrix} \rightarrow \text{Échange } L_1 \leftrightarrow L_3 \rightarrow \begin{pmatrix} \mathbf{6} & 7 & 2 & -1 \\ 4 & 4 & 1 & 2 \\ 2 & 1 & -1 & 3 \\ 2 & 5 & 3 & 4 \end{pmatrix}$$

Étape 2 : Élimination colonne 1

Multiplicateurs:

$$m_{21} = \frac{4}{6} = \frac{2}{3}$$
$$m_{31} = \frac{2}{6} = \frac{1}{3}$$
$$m_{41} = \frac{2}{6} = \frac{1}{3}$$

$$\begin{pmatrix} 6 & 7 & 2 & -1 \\ 0 & -\frac{2}{3} & -\frac{1}{3} & \frac{8}{3} \\ 0 & -\frac{4}{3} & -\frac{5}{3} & \frac{10}{3} \\ 0 & \frac{8}{3} & \frac{7}{3} & \frac{13}{3} \end{pmatrix}$$

Étape 3 : Recherche du pivot dans la colonne 2

Pivot maximal: $|a_{42}| = \frac{8}{3}$ (ligne 4)

$$\rightarrow \text{Échange } L_2 \leftrightarrow L_4 \rightarrow \begin{pmatrix} 6 & 7 & 2 & -1 \\ 0 & \mathbf{\frac{8}{3}} & \frac{7}{3} & \frac{13}{3} \\ 0 & -\frac{4}{3} & -\frac{5}{3} & \frac{10}{3} \\ 0 & -\frac{2}{3} & -\frac{1}{3} & \frac{8}{3} \end{pmatrix}$$

Étape 4 : Élimination colonne 2

Multiplicateurs:

$$m_{32} = \frac{-4/3}{8/3} = -\frac{1}{2}$$
$$m_{42} = \frac{-2/3}{8/3} = -\frac{1}{4}$$

$$\begin{pmatrix} 6 & 7 & 2 & -1 \\ 0 & \frac{8}{3} & \frac{7}{3} & \frac{13}{3} \\ 0 & 0 & -\frac{3}{2} & \frac{11}{2} \\ 0 & 0 & \frac{1}{4} & \frac{15}{4} \end{pmatrix}$$

Étape 5 : Recherche du pivot dans la colonne 3

Pivot maximal: $|a_{33}| = \frac{3}{2}$ (ligne 3)

$$\rightarrow \text{Échange } L_3 \leftrightarrow L_4 \rightarrow \begin{pmatrix} 6 & 7 & 2 & -1 \\ 0 & \frac{8}{3} & \frac{7}{3} & \frac{13}{3} \\ 0 & 0 & \mathbf{\frac{1}{4}} & \frac{15}{4} \\ 0 & 0 & -\frac{3}{2} & \frac{11}{2} \end{pmatrix}$$

Matrice triangulaire finale

$$U = \begin{pmatrix} 6 & 7 & 2 & -1 \\ 0 & \frac{8}{3} & \frac{7}{3} & \frac{13}{3} \\ 0 & 0 & \frac{1}{4} & \frac{15}{4} \\ 0 & 0 & 0 & -17 \end{pmatrix}, \quad \mathbf{c} = \begin{pmatrix} 3 \\ 2 \\ 2 \\ -17 \end{pmatrix}$$

Code Python commenté et expliqué

Importation et données

Objectif : Importer NumPy et définir les données du circuit.

```
# Importation du module Numpy
import numpy as np
# Donn es
Vg=80 # Tension du g n rateur
#Cr ation d'un tab des r sistances , cel  peut s'averrer utlise
    en cas de mofication de valeurs
R=np.array([2.5,10,5.1,8.2,7.9,3.6,3.3,3,7.1])*1000.0 # Tab des
   Resistances en Ohms
```

Note : Le `.0` est essentiel pour garantir que les résistances soient en `float`, évitant les divisions entières.

Construction de la matrice A et du vecteur b

Objectif : Définir les équations des mailles et des nœuds sous forme matricielle.

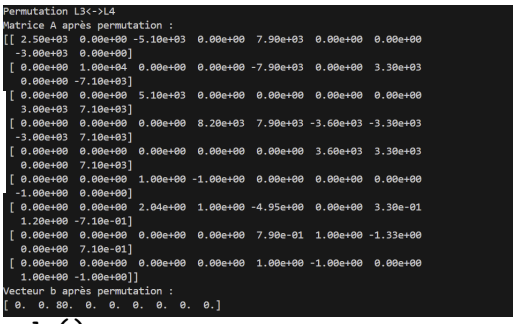
```
#MATRICE A 9x9
A = np.array([
    # Mailles(4  quations )
    [R[0],0,-R[2],0,R[4],0,0,-R[7],0],    # Maille 1
    [0,R[1],0,0,-R[4],0,R[6],0,-R[8]],    # Maille 2
    [0,R[1],0,R[3],0,-R[5],0,-R[7],0],    # Maille 3
    [R[0],R[1],0,0,0,0,R[6],0,0],         # Maille 4
    [0,0,0,0,0,R[5],R[6],0,R[8]],         # Maille 5
    # N uds(4  quations )
    [0,0,1,-1,0,0,0,-1,0],    # N ud A
    [1,-1,0,1,-1,0,0,0,0],    # N ud B
    [0,1,0,0,0,1,-1,0,0],     # N ud C
    [0,0,0,0,1,-1,0,1,-1]     # N ud D
])
# Matrice Col B
b=np.array([0,0,0,Vg,0,0,0,0,0])
```

```
Matrice A initiale
[[ 2.5e+03  0.0e+00 -5.1e+03  0.0e+00  7.9e+03  0.0e+00  0.0e+00 -3.0e+03
   0.0e+00]
 [ 0.0e+00  1.0e+04  0.0e+00  0.0e+00 -7.9e+03  0.0e+00  3.3e+03  0.0e+00
  -7.1e+03]
 [ 0.0e+00  1.0e+04  0.0e+00  8.2e+03  0.0e+00 -3.6e+03  0.0e+00 -3.0e+03
   0.0e+00]
 [ 2.5e+03  1.0e+04  0.0e+00  0.0e+00  0.0e+00  0.0e+00  3.3e+03  0.0e+00
   0.0e+00]
 [ 0.0e+00  0.0e+00  0.0e+00  0.0e+00  0.0e+00  3.6e+03  3.3e+03  0.0e+00
   7.1e+03]
 [ 0.0e+00  0.0e+00  1.0e+00 -1.0e+00  0.0e+00  0.0e+00  0.0e+00 -1.0e+00
   0.0e+00]
 [ 1.0e+00 -1.0e+00  0.0e+00  1.0e+00 -1.0e+00  0.0e+00  0.0e+00  0.0e+00
   0.0e+00]
 [ 0.0e+00  1.0e+00  0.0e+00  0.0e+00  0.0e+00  1.0e+00 -1.0e+00  0.0e+00
   0.0e+00]
 [ 0.0e+00  0.0e+00  0.0e+00  0.0e+00  1.0e+00 -1.0e+00  0.0e+00  1.0e+00
  -1.0e+00]]
det(A)= 1.0352300267999938e+20
rang(A)= 9
```

Fonction `gauss()`

Objectif : Transformer la matrice A en forme triangulaire supérieure par pivot partiel.

```
def gauss(A,b):
    A_copy=A.copy().astype(float)
    b_copy=b.copy().astype(float)
    for k in range(n):
        p=k
        maxi=abs(A_copy[k,k])
        for i in range(k+1,n):
            if abs(A_copy[i,k])>maxi:
                maxi=abs(A_copy[i,k])
                p=i
        if maxi<precision:
            print("Pivot nul")
            return None, None
        if p!=k:
            A_copy[[k,p]]=A_copy[[p,k]]
            b_copy[k], b_copy[p] = b_copy[p], b_copy[k]
            print(f"\nPermutation L{k+1}<->L{p+1}")
            print("Matrice A apr s permutation :")
            print(A_copy)
            print("Vecteur b apr s permutation :")
            print(b_copy)
        for i in range(k+1,n):
            h=A_copy[i,k]/A_copy[k,k]
            for j in range(k,n):
                A_copy[i,j]=A_copy[i,j]-h*A_copy[k,j]
            b_copy[i]=b_copy[i]-h*b_copy[k]
    return A_copy,b_copy
```



Fonction `retrosub()`

Objectif : Appliquer la rétro-substitution pour résoudre le système triangulaire.

```
def retrosub(A,b):
    n=len(b)
    X=np.zeros(n)
    for i in range(n-1,-1,-1):
        s=0
        for j in range(i+1, n):
            s=s+(A[i,j]*X[j])
        X[i]=(b[i]-s)/A[i,i]
    return X
```

Exécution finale

Objectif : Résoudre le système et afficher les intensités.

```
A_tri,b_prime=gauss(A,b)
if A_tri is not None:
    X=retrosub(A_tri,b_prime)
    print("COURANTS (mA):")
    for i in range(9):
        print(f"I{i+1}={X[i]*1000:} mA")

    print(f"\nCourant total: I1+I3={(X[0]+X[2])*1000:} mA")
    print(f"R sistance  quivalente : {Vg/(X[0]+X[2])/1000:} Kohm
       ")
```

```
Vecteur b après permutation :
[ 0.00000000e+00  0.00000000e+00  8.00000000e+01  0.00000000e+00
 -3.20000000e-02  0.00000000e+00 -4.27502578e-03 -2.08997206e-02
 -5.41142504e-03]
COURANTS (mA):
I1=15.807770231109766 mA
I2=8.470928946204328 mA
I3=-11.890190509686633 mA
I4=-17.778992188714597 mA
I5=-10.442150903809159 mA
I6=-21.873569867361187 mA
I7=-13.40264092115687 mA
I8=5.888801679027963 mA
I9=17.320220642579994 mA

Courant total: I1+I3=3.9175797214231323 mA
Résistance équivalente: 20.420771417240886 Kohm
```

Problèmes rencontrés

1. **Erreur d'indentation dans la fonction** `gauss()` : le test du pivot nul était mal placé, ce qui empêchait la détection correcte des pivots trop petits.
2. **Retour prématuré dans la boucle** : un `return` mal positionné pouvait interrompre l'algorithme après une seule étape.
3. **Type de données incorrect** : la matrice des résistances était initialement en `int`, ce qui provoquait des divisions entières et des erreurs de type. La solution a été de forcer les valeurs en `float` avec `1000.0` et `astype(float)`.
4. **Système singulier** : le rang de la matrice était parfois inférieur à 9, ce qui signifie que certaines équations étaient redondantes. Cela a été vérifié avec `np.linalg.matrix_rank()`.

VIII Complexité et coût opératoire

D'après le cours :

— Coût triangulation (Alg. 1.3) : $\approx \frac{2}{3}n^3$ opérations flottantes.

— Coût rétro-substitution (Alg. 1.2) : $\approx n^2$.

On a $n = 9$: triangulation $\approx \frac{2}{3} \cdot 9^3 = 486$ FLOPs, rétro-sub. ≈ 81 flops, total proche de 567 flops.

IX Résultats numériques

Résolution par `numpy.linalg.solve` et par les algos donnent les même solutions(Ce qui nous confirme que le code fonctionne bien sur des matrices de grandes tailles) :

Courant	Valeur (mA)
I_1	15.807770
I_2	8.470929
I_3	−11.890191
I_4	−17.778992
I_5	−10.442151
I_6	−21.873570
I_7	−13.402641
I_8	5.888802
I_9	17.320221

Courant fourni par le générateur (On utilise cette relation $I_{\text{gen}} = I_1 + I_3$) :

$$I_{\text{gen}} = 0.015807770 - 0.011890191 = 0.003917580 \text{ A} = 3.917580 \text{ mA}.$$

Résistance équivalente vue par le générateur

$$R_{\text{eq}} = \frac{V_g}{I_{\text{gen}}} \approx \frac{80}{0.003917580} \approx 20420.771 \ \Omega \approx 20.421 \ \text{k}\Omega.$$

X Conclusion

La résolution par le pivot de Gauss et la rétrosubstitution a permis d'obtenir les solutons des courants Ii du circuit. De plus la matricecest inversible (rang = n = 9) et les solutions obtenues par l'algorithme sont identiques à celles de `numpy.linalg.solve`. A noter que certaines étapes de calcul ont été sensibles au conditionnement de la matrice, ce qui a nécessité une attention particulière sur le choix des équations et d'éviter l'obtention de matrice non inversible.

Annexe : code Python documenté.

Références:

https://www.overleaf.com/learn/latex/Matrices
https://www.overleaf.com/learn/latex/Mathematical_expressions
https://github.com/Hazmatt101/Gaussian-Elimination-With-Partial-F
https://www.youtube.com/watch?v=9fJicFeM_CU&ab_channel=Lesmathsparl%27exemple
https://www.youtube.com/watch?v=1rjDgLRckH4&ab_channel=Lesmathsparl%27exemple